

Lecture Summary: Monte Carlo Tree Search (MCTS)

Overview

Monte Carlo Tree Search (MCTS) is a simulation-based planning method that combines *tree search* with *multi-armed bandit* ideas. It is designed for settings where:

- a simulator exists and can reset to arbitrary states,
- rewards are sparse (often only at terminal states),
- the branching factor is too large for minimax or α - β pruning,
- exhaustive search is infeasible.

MCTS adaptively focuses computation on the most promising trajectories and forms the core of modern game-playing systems such as MoGo, AlphaGo, AlphaZero, and MuZero.

Historical Motivation

Early successes in Computer Go revealed that:

- high branching factors make classical game-tree search intractable,
- playout (simulation)-based evaluation is effective when a simulator is available,
- bandit algorithms can guide search toward promising moves.

The key idea: *Tree Search + Bandits = MCTS*. Each node in the tree is treated like a bandit problem, and previously successful actions are explored more deeply.

Upper Confidence Bounds in Trees (UCT)

The most widely used MCTS variant is UCT (Kocsis & Szepesvári, 2006).

Goal. Find a path from the root to a leaf with highest expected reward in a tree of depth D and branching factor K .

UCT Score. At a node j , for each child $k = 1, \dots, K$, UCT selects the child maximizing

$$B_t(k) = \hat{\mu}_{k,t} + c \sqrt{\frac{\log(n_{j,t})}{n_{k,t}}}.$$

Here:

- $\hat{\mu}_{k,t}$ is the empirical mean reward of subtree k ,
- $n_{k,t}$ is the number of times child k has been visited,
- $n_{j,t}$ counts visits to node j ,
- $c > 0$ controls exploration.

This mirrors the classical UCB rule but is applied hierarchically across the tree.

Phases of the MCTS Algorithm

One iteration of MCTS consists of:

- 1. Selection.** Starting at the root, recursively choose the child with highest UCT score until reaching a leaf.
- 2. Expansion.** If the leaf corresponds to a previously visited state and actions remain unexplored, create new child nodes (often using a simulator or `deepcopy`).
- 3. Simulation (Rollout).** Perform a random or heuristic ployout from the expanded node until termination, generating a Monte-Carlo estimate of the value.

4. Backpropagation. Propagate the final reward back up the path, updating visit counts and empirical means.

Theoretical Guarantees

Consistency. As the number of simulations grows:

$$\hat{\mu}_{i,t} \rightarrow \mu_i \quad \text{for each leaf } i,$$

and UCT eventually selects the optimal path with probability 1.

Finite-Time Performance. Analysis is subtle because:

- leaf nodes behave like stationary bandits,
- internal nodes behave like *non-stationary* bandits (their estimates drift as deeper nodes become better understood),
- UCB analysis can be adapted to mild drift conditions.

A regret bound of order $O(K \log T)$ can be obtained, but additive constants may be *doubly exponential* in tree depth D . Thus finite-time performance may be extremely poor in worst-case trees.

Practical Behavior

In some adversarial tree structures, UCT may spend exponentially many simulations on suboptimal branches before discovering the optimal one. This is because UCB pulls suboptimal arms only $O(\log n)$ times, delaying discovery when high-reward leaves are deep and rare.

Modern Extensions

AlphaZero. Combines UCT with:

- a value network $V(s)$,

- a policy network $\pi(s, a)$,
- mixed scores that combine UCT bonus, neural value predictions, and policy priors,
- bootstrapping or full Monte-Carlo rollouts.

MuZero. Learns its own dynamics model to simulate rollouts in a latent space and uses value bounds to stabilize planning.

General Theme. MCTS scales to large state spaces by coupling:

- bandit-style exploration,
- function approximation (values, policies, or dynamics),
- selective lookahead search.

It remains the foundation for state-of-the-art reasoning and planning systems.